

agent: discovery-architecture-design-agent cli: Claude Code CLI llm: claude-opus-4-6 run_id: 20260814T163333_cchq5z generated_at: 2026-08-14T11:03:33.087Z

1. Architecture & Design Hotspots Analysis

Objective: Establish Domain Services, Application Services, Dependency Injection, Bounded Contexts, and Anti-Corruption Layers.

Date: 2026-08-14 16:35:28 IST | **Scope:** shende-shweta/FSDKC (main) — PHP 8.3 / Laravel 12 monolithic REST API (backend), React 19 / TypeScript / Vite 6 SPA (frontend), Node.js / Express dev-API, MariaDB 11, MongoDB 7, Docker / Nginx / PHP-FPM, GitHub Actions CI

Layers analyzed: Backend (26 PHP files, 1 127 LOC), Frontend (15 TSX/JSX/TS files, 983 LOC), Dev-API (6 JS files, 832 LOC). Total: 47 source files, 2 942 LOC.

Executive Summary

EXECUTIVE SUMMARY

The Klearcom platform is a voice-quality observability system built as a Laravel 12 monolithic REST API with a React 19 SPA frontend and a parallel Node.js dev-API. While controllers are commendably thin (avg 74 LOC) and the frontend properly uses a shared API client with React Query, the codebase has three systemic architecture issues: (1) zero repository abstractions — all 32 database access points scatter Eloquent calls through controllers and services; (2) domain boundary violations between the Discovery and Connect modules, which share MongoDB collections and cross-read each other's models without interfaces; and (3) critical business logic (the reachability KPI formula) is duplicated in four independent locations across two codebases with no canonical source of truth. The `app/Modules/Connect` and `app/Modules/Discovery` skeleton directories signal an intent to modularize that has not yet begun. These gaps collectively create high change-amplification risk — modifying the reachability calculation, swapping a persistence layer, or extracting a bounded context will each require hunting across multiple files and two languages.

6

CONTROLLERS / HANDLERS

4

MODELS / ENTITIES

2

SERVICE CLASSES FOUND

0

REPOSITORY CLASSES
FOUND

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Repository Pattern (H3), Domain Boundary Violations (H8), Shared Database Coupling (H9), and Duplicated Business Logic (H10).

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	74 LOC	GOOD
H2	Missing Service Layer	Controller methods directly accessing models	<10	10–20	>20	13 methods	MODERATE
H3	Missing Repository Pattern	Direct DB/ORM access points outside repositories	<10	10–20	>20	32	HIGH RISK
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	1 (LegacyDataMapper)	MODERATE
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	100%	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	6	HIGH RISK
H9	Shared Database Coupling	Data stores shared across domains	<10%	10–30%	>30%	37.5% (3/8)	HIGH RISK
H10	Duplicated Business Logic (additional)	Duplicated business-critical code sites	0–1	2–4	>4	7 sites	HIGH RISK
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	86 LOC	GOOD
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	2	GOOD

F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	≤2 levels	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2	MODERATE
C1	Anemic Domain Model (context)	Models with domain behaviour (% of total)	>70%	30–70%	<30%	0% (0/4)	HIGH RISK
C2	Empty Module Skeleton (context)	Module directories with actual code	100%	50–100%	<50%	0% (0/2)	HIGH RISK

Context-driven items owned by other reports:

- Missing Authentication/Authorization (zero auth middleware on all API routes) → owned by `06-security.md`.
- Unsafe `extract()` usage in LegacyDataMapper → owned by `06-security.md` and `03-code-quality.md`.
- No Laravel migrations present (schema managed via Docker init.sql) → owned by `07-technical-debt.md`.

No additional hotspots beyond the standard set and context items were observed.

1.2 Hotspot-by-Hotspot Evidence

H2. Missing Service Layer MEDIUM

Benchmark: `Controller methods directly accessing models = 13` → falls in the **Moderate** band (Good <10 · Moderate 10–20 · High Risk >20).

What to check: Controllers/handlers directly accessing repositories/models with no dedicated service tier.

Evidence: 13 of 20 controller methods access Eloquent models directly instead of delegating to a service. Only `MongoController` (3 methods) and `StreamController` (3 methods) properly delegate to `MongoService`.

Example 1 — `DashboardController::kpis` performs 8 Eloquent queries inline:

```
// backend/app/Http/Controllers/Api/DashboardController.php:14-33
$discoveryTotal = DiscoveryJob::count();
$discoveryCompleted = DiscoveryJob::where('status', 'completed')->count();
$connectMonitors = ConnectMonitor::count();
$avgReachability = ConnectMonitor::avg('reachability_pct') ?? 0;
```

```
$alerts = ConnectMonitor::where('status', 'alert')->count();
// ... plus 3 more inline queries at lines 30-33
```

Example 2 — `ConnectController::checks` embeds a reachability calculation with 2 extra queries:

```
// backend/app/Http/Controllers/Api/ConnectController.php:66-75
$recentChecks = ConnectCheckResult::where('connect_monitor_id', $id)
    ->orderByDesc('checked_at')
    ->limit(20)
    ->get();
$successRate = $recentChecks->count() > 0
    ? ($recentChecks->where('reachable', true)->count() / $recentChecks-
    >count()) * 100
    : 100;
```

Example 3 — `LegacyReportController::carrierSummary` loops through monitors and queries check results per monitor inside the loop (N+1 pattern):

```
// backend/app/Http/Controllers/Api/LegacyReportController.php:33-41
foreach ($monitors as $monitor) {
    $recent = ConnectCheckResult::where('connect_monitor_id', $monitor->id)
        ->orderByDesc('checked_at')
        ->limit(20)
        ->get();
    $successRate = $recent->count() > 0
        ? ($recent->where('reachable', true)->count() / $recent->count()) * 100
        : 100;
```

Total: 13 controller methods with direct model access across 4 controllers.

Why it matters here: KPI aggregation logic in `DashboardController`, reachability computation in `ConnectController` and `LegacyReportController`, and tree-building in `DiscoveryController` are all business rules that cannot be reused from CLI commands, queued jobs, or the dev-API — each caller must re-implement them. The N+1 loop in `LegacyReportController::carrierSummary` would be the first performance bottleneck as monitor count grows.

Recommended approach:

1. Create a `ConnectService` that owns the reachability calculation, check history retrieval, and monitor management.
2. Create a `DiscoveryService` that owns the tree-building logic and job lifecycle.
3. Create a `DashboardService` that composes KPIs from the other services.
4. Keep controllers as thin HTTP → Service → Response translators.

4 files affected.

File	Line(s)	Issue	Action
<code>backend/app/Http/Controllers/Api/ConnectController.php</code>	22, 36, 47, 59, 60, 66, 89	Direct model access in controller	Extract to service class

<code>backend/app/Http/Controllers/Api/DashboardController.php</code>	14, 15, 16, 17, 18, 30, 31, 33	Direct model access in controller	Extract to service class
<code>backend/app/Http/Controllers/Api/DiscoveryController.php</code>	21, 35, 46, 57, 58, 69	Direct model access in controller	Extract to service class
<code>backend/app/Http/Controllers/Api/LegacyReportController.php</code>	24, 34, 58, 59	Direct model access in controller	Extract to service class

H3. Missing Repository Pattern CRITICAL

Benchmark: Direct DB/ORM access points outside repositories = 32 → falls in the **High Risk** band (Good <10 · Moderate 10–20 · High Risk >20).

What to check: Direct DB/ORM access scattered through the codebase without a repository abstraction layer.

Evidence: Zero repository classes exist. All 32 Eloquent access points are spread across 4 controllers and 1 service. The `MongoService` acts as a de facto repository for MongoDB (8 methods), but no equivalent exists for MariaDB/Eloquent.

Example 1 — `ConnectController` performs 7 direct model calls across 5 methods:

```
// backend/app/Http/Controllers/Api/ConnectController.php:22
$monitors = ConnectMonitor::orderByDesc('last_checked_at')->get();

// backend/app/Http/Controllers/Api/ConnectController.php:60-68
$checks = ConnectCheckResult::where('connect_monitor_id', $id)
    ->orderByDesc('checked_at')
    ->limit(50)
    ->get();
$recentChecks = ConnectCheckResult::where('connect_monitor_id', $id)
    ->orderByDesc('checked_at')
    ->limit(20)
    ->get();
```

Example 2 — `RealTimeTestService` has 6 direct model calls, mixing Eloquent reads and writes:

```
// backend/app/Services/RealTimeTestService.php:24-25
$job = DiscoveryJob::findOrFail($jobId);
$job->update(['status' => 'running', 'started_at' => now()]);

// backend/app/Services/RealTimeTestService.php:52-58
$node = DiscoveryNode::create([
    'discovery_job_id' => $jobId,
    'parent_id' => $parentId,
    'prompt_text' => $step['transcript'] ?? 'Menu discovered',
    'node_type' => 'menu',
    'depth' => $parentId ? 1 : 0,
]);
```

Example 3 — `DashboardController::kpis` queries 4 different models with 8 Eloquent calls, none via a repository:

```
// backend/app/Http/Controllers/Api/DashboardController.php:14-18
$discoveryTotal = DiscoveryJob::count();
$discoveryCompleted = DiscoveryJob::where('status', 'completed')->count();
$connectMonitors = ConnectMonitor::count();
$avgReachability = ConnectMonitor::avg('reachability_pct') ?? 0;
$alerts = ConnectMonitor::where('status', 'alert')->count();
```

Breakdown: ConnectController (7), DashboardController (8), DiscoveryController (6), LegacyReportController (5), RealTimeTestService (6) = 32 total.

Why it matters here: Every query is hard-coded to Eloquent's MySQL driver. If the team ever needs to swap MariaDB for Aurora, add read replicas, or cache hot queries (e.g. the dashboard KPI aggregation that fires on every page load), there is no seam to intercept. Testing any controller requires a live database — mocking 32 scattered static calls is impractical.

Recommended approach:

1. Create `ConnectMonitorRepository` and `ConnectCheckResultRepository` interfaces under `app/Repositories/`.
2. Create `DiscoveryJobRepository` and `DiscoveryNodeRepository` interfaces.
3. Implement concrete Eloquent repositories, bind via `AppServiceProvider`.
4. Inject repositories into services (not controllers) — controllers call services, services call repositories.

5 files affected.

File	Line(s)	Issue	Action
<code>backend/app/Http/Controllers/Api/ConnectController.php</code>	22, 36, 47, 59, 60, 66, 89	Direct ORM access without repository	Wrap in repository interface
<code>backend/app/Http/Controllers/Api/DashboardController.php</code>	14, 15, 16, 17, 18, 30, 31, 33	Direct ORM access without repository	Wrap in repository interface
<code>backend/app/Http/Controllers/Api/DiscoveryController.php</code>	21, 35, 46, 57, 58, 69	Direct ORM access without repository	Wrap in repository interface
<code>backend/app/Http/Controllers/Api/LegacyReportController.php</code>	24, 34, 58, 59	Direct ORM access without repository	Wrap in repository interface
<code>backend/app/Services/RealTimeTestService.php</code>	24, 52, 67, 72, 83, 108, 117	Direct ORM access without repository	Wrap in repository interface

H5. Shared Utility Abuse LOW

Benchmark: `Utility files with business logic = 1` → falls in the **Moderate** band (Good 0 · Moderate 1–5 · High Risk >5).

What to check: Large utility/helper files used everywhere, holding business logic.

Evidence: `LegacyDataMapper` sits in `app/Legacy/` and uses the unsafe `extract()` pattern to transform business data.

```
// backend/app/Legacy/LegacyDataMapper.php:10-20
public function mapReportRow(array $row): array
{
    extract($row, EXTR_SKIP);
```

```

return [
    'label' => $name ?? 'Unknown',
    'metric' => $reachability_pct ?? 0,
    'region' => $country_code ?? 'N/A',
    'source' => 'legacy_extract_mapper',
];
}

```

It is instantiated directly (not via DI) in `LegacyReportController::carrierSummary` (line 31: `$mapper = new LegacyDataMapper()`), bypassing the container entirely.

Why it matters here: The `extract()` call creates variables from arbitrary arrays, which is a maintainability hazard (impossible to trace variable origins via static analysis) and a security concern (potential variable injection if the source array is user-influenced — and in `LegacyReportController` it is, since `extract($filters)` at line 22 extracts from `$request->all()`). The `new LegacyDataMapper()` instantiation means this class cannot be easily mocked or substituted.

Recommended approach:

1. Replace `extract()` with explicit array access (`$row['name']`).
2. Register `LegacyDataMapper` as a service in the container and inject via constructor DI.
3. Move the mapping logic into a dedicated `ReportTransformer` or DTO.

2 files affected.

File	Line(s)	Issue	Action
<code>backend/app/Http/Controllers/Api/LegacyReportController.php</code>	22	Unsafe extract() pattern	Replace with explicit array access
<code>backend/app/Legacy/LegacyDataMapper.php</code>	6, 12, 24	Unsafe extract() pattern	Replace with explicit array access

H8. Domain Boundary Violations HIGH

Benchmark: `Cross-domain access points = 6` → falls in the **High Risk** band (Good 0 · Moderate 1–5 · High Risk >5).

What to check: Code in one business area directly reading/writing another area's data or models.

Evidence: Two business domains exist — **Discovery** (DiscoveryJob, DiscoveryNode) and **Connect** (ConnectMonitor, ConnectCheckResult). Several components cross these boundaries without interfaces or anti-corruption layers.

Example 1 — `DashboardController` reads directly from both domains (8 Eloquent calls spanning both):

```

// backend/app/Http/Controllers/Api/DashboardController.php:14-18
$discoveryTotal = DiscoveryJob::count();           // Discovery domain
$discoveryCompleted = DiscoveryJob::where('status', 'completed')->count(); //
Discovery domain
$connectMonitors = ConnectMonitor::count();       // Connect domain
$avgReachability = ConnectMonitor::avg('reachability_pct') ?? 0; // Connect
domain

```

Example 2 — `LegacyReportController` spans both domains: `carrierSummary` reads Connect models, `ivrDepthReport` reads Discovery models — both in the same controller class:

```
// backend/app/Http/Controllers/Api/LegacyReportController.php:24-27 (Connect)
$monitors = ConnectMonitor::query()
    ->when(isset($country_code), fn ($q) => $q->where('country_code',
$country_code))

// backend/app/Http/Controllers/Api/LegacyReportController.php:58-59
(Discovery)
$job = DiscoveryJob::findOrFail($jobId);
$nodes = DiscoveryNode::where('discovery_job_id', $jobId)->get();
```

Example 3 — **RealTimeTestService** handles both Discovery test execution and Connect test execution in a single class, directly accessing models from both domains:

```
// backend/app/Services/RealTimeTestService.php:24 (Discovery)
$job = DiscoveryJob::findOrFail($jobId);

// backend/app/Services/RealTimeTestService.php:83 (Connect)
$monitor = ConnectMonitor::findOrFail($monitorId);
```

Cross-domain access points: DashboardController (2 patterns), LegacyReportController (2 patterns), RealTimeTestService (2 patterns) = 6 total.

Additionally, the empty **app/Modules/Connect/** and **app/Modules/Discovery/** directories (containing only **AGENTS.md** placeholders) show the team has recognized the need for bounded contexts but has not yet moved any code into the module structure.

Why it matters here: Extracting either Discovery or Connect into an independent deployable (microservice, package, or modular monolith boundary) would require untangling every cross-domain call. The **DashboardController** is the most entangled — it would need to call each domain's published interface rather than reaching into their models. The **RealTimeTestService** conflates two distinct workflows (IVR traversal vs. TFN reachability) in one class, making it the highest-risk component for unintended regressions.

Recommended approach:

1. Split **RealTimeTestService** into **DiscoveryTestService** and **ConnectTestService**.
2. Create **DiscoveryQueryService** and **ConnectQueryService** with published interfaces for cross-domain reads.
3. **DashboardController** and **LegacyReportController** should depend on these query interfaces, not on the raw models.
4. Move domain code into **app/Modules/Connect/** and **app/Modules/Discovery/** to enforce the boundaries structurally.

1 file affected.

File	Line(s)	Issue	Action
backend/app/Http/Controllers/Api/DashboardController.php	14, 15, 16, 17, 18, 30, 31, 33	Cross-domain model access	Replace with domain query interface

1 file affected.

File	Line(s)	Issue	Action
------	---------	-------	--------

<code>backend/app/Http/Controllers/Api/LegacyReportController.php</code>	24, 34, 58, 59	Cross-domain model access	Replace with domain query interface
--	----------------	---------------------------	-------------------------------------

1 file affected.

File	Line(s)	Issue	Action
<code>backend/app/Services/RealTimeTestService.php</code>	24, 52, 67, 72, 83, 108, 117	Multi-domain service coupling	Split into domain-specific services

H9. Shared Database Coupling HIGH

Benchmark: Data stores shared across domains = 37.5% (3 of 8) → falls in the **High Risk** band (Good <10% · Moderate 10–30% · High Risk >30%).

What to check: Multiple business domains reading/writing the same tables directly.

Evidence: The application uses 8 data stores: 5 MariaDB tables (domain-owned) and 3 MongoDB collections (shared). The MariaDB schema is cleanly separated:

Table	Owner Domain
<code>users</code>	General (no FK to any domain)
<code>discovery_jobs</code>	Discovery
<code>discovery_nodes</code>	Discovery
<code>connect_monitors</code>	Connect
<code>connect_check_results</code>	Connect

However, all 3 MongoDB collections are shared by both domains via a `module` field discriminator:

Collection	Writers	Discriminator
<code>transcripts</code>	Discovery, Connect	<code>module</code> field
<code>test_events</code>	Discovery, Connect	<code>module</code> field
<code>call_diagnostics</code>	Discovery, Connect	<code>module</code> field

Both `RealTimeTestService::runDiscoveryTest` and `RealTimeTestService::runConnectTest` write to the same collections via `MongoService` :

```
// backend/app/Services/RealTimeTestService.php:36-38 (Discovery writing to shared collection)
$this->mongo->storeTestEvent($sessionId, 'discovery', $jobId, [...]);

// backend/app/Services/RealTimeTestService.php:94 (Connect writing to same collection)
$this->mongo->storeTestEvent($sessionId, 'connect', $monitorId, [...]);
```

The MongoDB `init.js` and `seed-data.js` confirm this shared design — both modules' seed data goes into the same collections with indexes only on `module + reference_id`.

Why it matters here: Schema changes to any shared collection (adding fields, changing indexes, modifying the document shape) affect both Discovery and Connect. There is no per-domain ownership of the MongoDB data model. If one domain needs a different transcript format or retention policy, the change risks breaking the other domain's reads. The `module` field is a soft discriminator with no schema enforcement.

Recommended approach:

1. Define per-domain MongoDB collection namespaces (e.g. `discovery_transcripts` , `connect_transcripts`) or logically separate via the module field with per-domain typed accessors.
2. Split `MongoService` into `DiscoveryMongoRepository` and `ConnectMongoRepository` with domain-specific document schemas.
3. If shared collections must remain, introduce an anti-corruption layer that maps domain-specific DTOs to/from the shared document format.

1 file affected.

File	Line(s)	Issue	Action
<code>backend/app/Services/MongoService.php</code>	62, 78, 95, 111, 125, 139	Shared MongoDB collections across domains	Split into domain-owned repositories

H10. Duplicated Business Logic (additional) CRITICAL

Benchmark: `Duplicated business-critical code sites = 7` → falls in the **High Risk** band (Good 0–1 · Moderate 2–4 · High Risk >4). Justification: business-critical KPI calculations duplicated across independent codebases compound the risk of divergence and silent correctness bugs; threshold is set tighter than generic code duplication.

What to check: Identical business logic implementations scattered across the codebase with no single source of truth.

Evidence: Two distinct business formulas are duplicated across 7 independent locations in 2 languages:

Reachability KPI formula — the same `successRate = reachableCount / totalCount * 100` with a `< 90` → `alert` threshold appears in 4 places:

Location 1:

```
// backend/app/Http/Controllers/Api/ConnectController.php:71-75
$successRate = $recentChecks->count() > 0
    ? ($recentChecks->where('reachable', true)->count() / $recentChecks->count()) * 100
    : 100;
$computedStatus = $successRate < 90 ? 'alert' : 'active';
```

Location 2:

```
// backend/app/Services/RealTimeTestService.php:118-122
$rate = $recent->count() > 0 ? ($recent->where('reachable', true)->count() / $recent->count()) * 100 : 100;
$monitor->update([
    'reachability_pct' => round($rate, 2),
    'status' => $rate < 90 ? 'alert' : 'active',
]);
```

Location 3:

```
// backend/app/Http/Controllers/Api/LegacyReportController.php:39-41
$successRate = $recent->count() > 0
    ? ($recent->where('reachable', true)->count() / $recent->count()) * 100
    : 100;
```

Location 4:

```
// dev-api/src/realtime.js:147-152
const successRate = recent.length > 0
    ? (recent.filter((c) => c.reachable).length / recent.length) * 100
    : 100;
monitor.reachability_pct = Math.round(successRate * 100) / 100;
monitor.status = successRate < 90 ? 'alert' : 'active';
```

IVR Tree-building algorithm — the recursive `buildTree(nodes, parentId)` function is duplicated in 3 places:

```
// backend/app/Http/Controllers/Api/DiscoveryController.php:87-101
private function buildTree($nodes, ?int $parentId = null): array { ... }

// backend/app/Http/Controllers/Api/LegacyReportController.php:78-92 (docblock:
"Duplicate of DiscoveryController::buildTree – copy-paste debt")
private function buildTree($nodes, ?int $parentId = null): array { ... }
```

```
// dev-api/src/store.js:64-75
export function buildTree(nodes, parentId = null) { ... }
```

Why it matters here: The reachability formula is the most business-critical KPI in the platform — it determines the `alert` status shown to operations users. If the 90% threshold or the calculation method is updated in one location but not the other three, monitors will show inconsistent alert states depending on which code path produced the value. The `ConnectController::checks` comment at line 65 explicitly acknowledges this duplication ("Duplicate reachability calculation block") but does not resolve it.

Recommended approach:

1. Extract `ReachabilityCalculator::compute(Collection $checks): ReachabilityResult` as a domain value object encapsulating the formula, sample size (20), and alert threshold (90%).
2. All 3 PHP locations call this single calculator; the dev-API imports the threshold as a constant.
3. Extract `IvrTreeBuilder::build(Collection $nodes): array` as a shared service used by both `DiscoveryController` and `LegacyReportController`.

5 files affected.

File	Line(s)	Issue	Action
<code>backend/app/Http/Controllers/Api/ConnectController.php</code>	71, 72, 75, 81	Duplicated business logic	Extract canonical service/value object

<code>backend/app/Http/Controllers/Api/DashboardController.php</code>	26, 27	Duplicated business logic	Extract canonical service/value object
<code>backend/app/Http/Controllers/Api/DiscoveryController.php</code>	63, 87, 97	Duplicated business logic	Extract canonical service/value object
<code>backend/app/Http/Controllers/Api/LegacyReportController.php</code>	39, 40, 46, 61, 77, 78, 88	Duplicated business logic	Extract canonical service/value object
<code>backend/app/Services/RealTimeTestService.php</code>	118	Duplicated business logic	Extract canonical service/value object

F5. Legacy / Inconsistent Component Patterns LOW

Benchmark: `Legacy-pattern components = 2` → falls in the **Moderate** band (Good 0 · Moderate 1–10 · High Risk >10).

What to check: Mixed paradigms, missing error boundaries, deprecated lifecycle/APIs, no shared component conventions.

Evidence: The frontend uses two distinct component paradigms:

Example 1 — `LegacyMonitorPoller` is the only React class component in a codebase that otherwise uses function components + hooks exclusively. It also has an intentional interval leak (no `componentWillUnmount`):

```
// frontend/src/components/LegacyMonitorPoller.jsx:17-33
export default class LegacyMonitorPoller extends Component<Props, State> {
  intervalId: ReturnType<typeof setInterval> | null = null;
  state: State = { reachability: null, error: null };

  componentDidMount() {
    this.intervalId = setInterval(() => {
      api.get<{ computed?: { reachability_pct: number } }>(...).then(...)
        .catch(...);
    }, 3000);
    // Intentionally no componentWillUnmount – EventSource/interval leak for
    audit finding
  }
```

Example 2 — `LegacyDashboardWidget` uses raw `useEffect` + `useState` for data fetching with manual `setInterval` polling, while every other page uses React Query (`useQuery` + `useMutation`). It also throws an uncaught error with no Error Boundary wrapper:

```
// frontend/src/pages/LegacyDashboardWidget.tsx:16-48
useEffect(() => {
  let cancelled = false;
  Promise.all([
    api.get<DashboardKpis>('/dashboard/kpis'),
    api.get<{ data: DiscoveryJob[] }>('/discovery/jobs'),
    api.get<{ data: ConnectMonitor[] }>('/connect/monitors'),
  ]).then(...).catch((err: Error) => {
    if (!cancelled) setError(err.message);
  });
}, [/* dependencies */]);
```

```
});
const timer = setInterval(() => {
  api.get<DashboardKpis>('/dashboard/kpis').then((k) => { ... });
}, 10000);
return () => { cancelled = true; clearInterval(timer); };
}, []);
// ...
if (error) throw new Error(error); // Uncaught – no Error Boundary wraps this
component
```

Why it matters here: A new developer seeing `LegacyMonitorPoller` might assume class components are acceptable in this codebase; the interval leak will cause memory issues in long-running SPA sessions. `LegacyDashboardWidget`'s `throw new Error(error)` will crash the entire React tree since no Error Boundary exists in the component hierarchy.

Recommended approach:

1. Convert `LegacyMonitorPoller` to a function component using the existing `useRealtimeTest` hook or a simpler `useQuery` with `refetchInterval`.
2. Convert `LegacyDashboardWidget` to use React Query (matching `DashboardPage`'s pattern).
3. Add an `ErrorBoundary` wrapper at the route or layout level.

2 files affected.

File	Line(s)	Issue	Action
<code>frontend/src/components/LegacyMonitorPoller.jsx</code>	17, 22, 32	Legacy component pattern	Convert to function component with hooks
<code>frontend/src/pages/LegacyDashboardWidget.tsx</code>	48	Legacy component pattern	Convert to function component with hooks

C1. Anemic Domain Model (context) HIGH

Benchmark: `Models with domain behaviour = 0% (0 of 4)` → falls in the **High Risk** band (Good >70% · Moderate 30–70% · High Risk <30%). Justification: an anemic domain model pushes all business logic into services and controllers, removing the natural enforcement point for invariants; above 70% of models having at least one non-trivial method is "Good" because it indicates domain rules are co-located with data.

What to check: Whether Eloquent models contain any domain behaviour (business methods, computed properties, validation rules, state transitions) or are purely data containers.

Evidence: All 4 Eloquent models are pure data containers with only `$fillable`, `$casts`, and relationship declarations — zero business methods:

```
// backend/app/Models/ConnectMonitor.php (entire file, 29 LOC)
class ConnectMonitor extends Model
{
  protected $fillable = ['name', 'toll_free_number', 'country_code',
'carrier', 'status', 'reachability_pct', 'last_checked_at'];
  protected $casts = ['reachability_pct' => 'float', 'last_checked_at' =>
'datetime'];
  public function checkResults(): HasMany { return $this-
```

```
>hasMany(ConnectCheckResult::class); }
}
```

The reachability calculation, alert-status determination, and tree-building logic all belong to the domain but live in controllers and services. For example, the `< 90 → alert` threshold is a domain invariant of `ConnectMonitor` that should be a model method like `updateReachability(Collection $recentChecks)`.

```
// backend/app/Models/DiscoveryJob.php (entire file, 32 LOC)
class DiscoveryJob extends Model
{
    protected $fillable = ['name', 'phone_number', 'country_code', 'status',
    ...];
    protected $casts = ['languages' => 'array', 'started_at' => 'datetime',
    'completed_at' => 'datetime'];
    public function nodes(): HasMany { return $this-
    >hasMany(DiscoveryNode::class); }
}
```

State transitions (`pending → running → completed`) are managed externally in `RealTimeTestService` via raw `$job->update(['status' => ...])` calls, with no validation that the transition is legal.

Why it matters here: Without domain methods, the business rules governing status transitions, reachability thresholds, and tree integrity are scattered across services and controllers. Any new developer adding a status change (e.g. `failed → retrying`) must discover all the places where status is set rather than having a single `DiscoveryJob::transitionTo(string $status)` method that enforces valid transitions.

Recommended approach:

1. Add `ConnectMonitor::computeReachability(Collection $checks): float` and `ConnectMonitor::isAlert(): bool`.
2. Add `DiscoveryJob::transitionTo(string $status): void` with valid-transition enforcement.
3. Add `DiscoveryNode::buildTree(Collection $siblings): array` as a model-level concern.
4. Treat models as the natural home for single-entity invariants; keep multi-entity orchestration in services.

4 files affected.

File	Line(s)	Issue	Action
<code>backend/app/Models/ConnectCheckResult.php</code>	8	Anemic domain model	Add domain behaviour methods
<code>backend/app/Models/ConnectMonitor.php</code>	8	Anemic domain model	Add domain behaviour methods
<code>backend/app/Models/DiscoveryJob.php</code>	8	Anemic domain model	Add domain behaviour methods
<code>backend/app/Models/DiscoveryNode.php</code>	9	Anemic domain model	Add domain behaviour methods

C2. Empty Module Skeleton (context) MEDIUM

Benchmark: `Module directories with actual code = 0% (0 of 2)` → falls in the **High Risk** band (Good 100% · Moderate 50–100% · High Risk <50%). Justification: skeleton directories that never receive code create false expectations of modular structure; 100% means every declared module has actual source.

What to check: Whether the `app/Modules/` directory structure contains actual domain code or is only a placeholder.

Evidence: Both backend and frontend have module directories, but all four contain only `AGENTS.md` placeholder files:

```
backend/app/Modules/Connect/AGENTS.md    (807 bytes)
backend/app/Modules/Discovery/AGENTS.md  (870 bytes)
frontend/src/modules/Connect/AGENTS.md   (320 bytes)
frontend/src/modules/Discovery/AGENTS.md (320 bytes)
```

Meanwhile, the actual Connect and Discovery code lives in the flat `app/Http/Controllers/Api/`, `app/Models/`, `app/Services/`, `frontend/src/pages/`, and `frontend/src/components/` directories with no structural separation.

Why it matters here: A developer inspecting the project structure sees `app/Modules/Connect/` and `app/Modules/Discovery/`, signalling that bounded contexts are in place — but clicking into them reveals no code. This creates a misleading architecture narrative. The empty skeleton also indicates a stalled modularization effort that should either be completed or removed to avoid confusion.

Recommended approach:

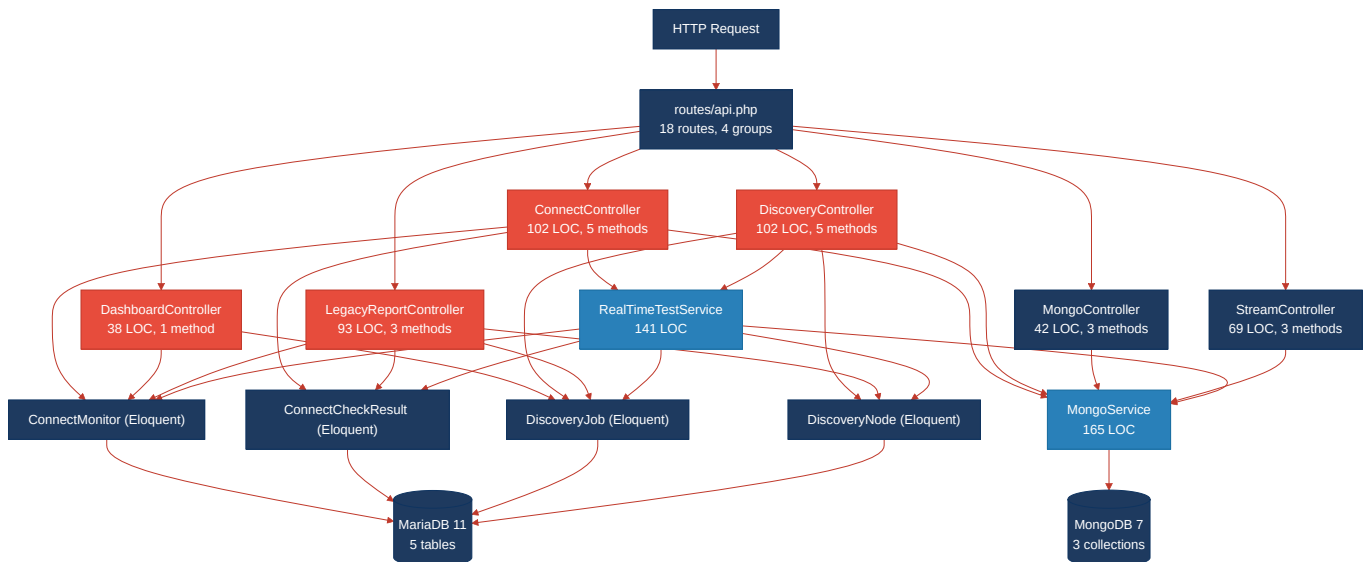
- Option A (Complete):** Move domain-specific controllers, services, models, and routes into `app/Modules/Connect/` and `app/Modules/Discovery/` with proper Laravel module structure (each with its own `routes/`, `Models/`, `Services/`, `Controllers/`).
- Option B (Remove):** Delete the empty `Modules/` directories if modularization is not on the near-term roadmap, and add them back when actual code migration begins.

No files matched `backend/app/Modules/**/*` containing `AGENTS.md`.

Not observed (rated Good): H1 (Fat Controllers — avg 74 LOC, all under 150), H4 (Circular Dependencies — no cycles detected; dependency flow is Controllers → Services → Models, no reverse edges), H6 (Direct SQL in Controllers — 100% ORM compliance, no raw SQL strings), H7 (God Classes — largest file is MongoService at 165 LOC, well under 1 000), F1 (Business Logic in Components — avg 86 LOC per component), F2 (Missing Frontend Service/Data Layer — only 2 legacy components bypass React Query; all others use the shared `api` client via `useQuery` / `useMutation`), F3 (God / Oversized Components — largest is ConnectPage at 222 LOC, under 400), F4 (Prop Drilling / Global State Abuse — Zustand store has only 4 fields; max prop depth ≤ 2 levels).

1.3 Diagrams

Current-state architecture (as-is)

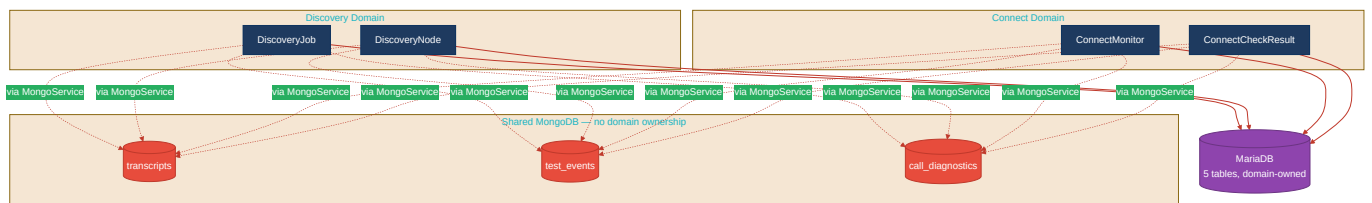


Clean reference path (target pattern found in codebase)

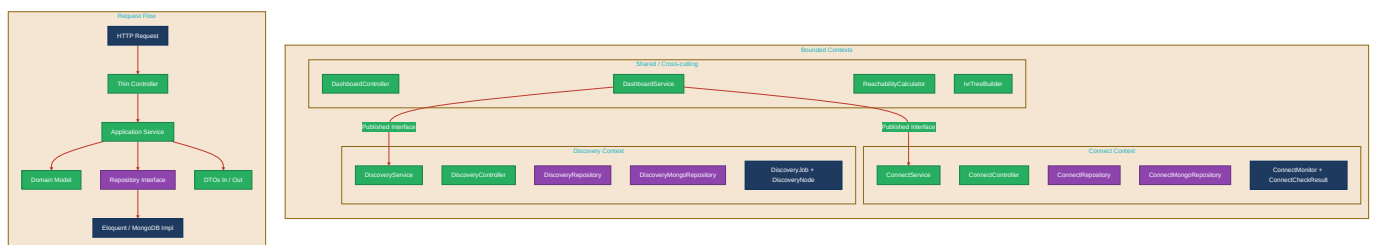
The **MongoController → MongoService → MongoDB** path is the cleanest architecture pattern in this codebase: a thin controller delegating entirely to an injected service.



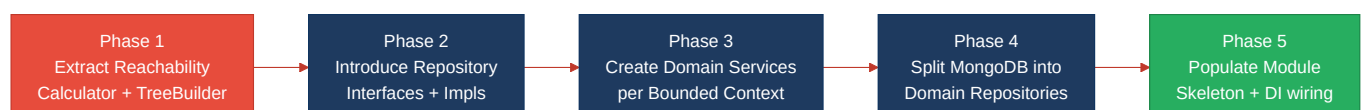
Domain boundary map (business domains vs. shared data)



Target architecture (proposed)



Improvement roadmap



1.4 Actions Required

Hotspot	Action	Rating	Priority
H3. Missing Repository Pattern	Introduce <code>ConnectMonitorRepository</code> , <code>ConnectCheckResultRepository</code> , <code>DiscoveryJobRepository</code> , <code>DiscoveryNodeRepository</code> interfaces with Eloquent implementations; bind via <code>AppServiceProvider</code> ; inject into services	HIGH RISK	CRITICAL
H10. Duplicated Business Logic	Extract <code>ReachabilityCalculator</code> value object and <code>IvrTreeBuilder</code> service as single sources of truth; update all 7 call sites across PHP and JS codebases	HIGH RISK	CRITICAL
H8. Domain Boundary Violations	Split <code>RealTimeTestService</code> into <code>DiscoveryTestService</code> and <code>ConnectTestService</code> ; create published query interfaces for cross-domain reads; refactor <code>DashboardController</code> and <code>LegacyReportController</code> to use them	HIGH RISK	HIGH
H9. Shared Database Coupling	Split shared MongoDB collections into domain-owned repositories with typed document schemas; or add anti-corruption layer between domains and shared collections	HIGH RISK	HIGH
C1. Anemic Domain Model	Add domain behaviour to models: <code>ConnectMonitor::computeReachability()</code> , <code>DiscoveryJob::transitionTo()</code> , <code>DiscoveryNode::buildTree()</code> ; enforce state-machine transitions	HIGH RISK	HIGH
H2. Missing Service Layer	Create <code>ConnectService</code> , <code>DiscoveryService</code> , <code>DashboardService</code> ; move all business logic out of 13 controller methods into services	MODERATE	MEDIUM
C2. Empty Module Skeleton	Either populate <code>app/Modules/Connect/</code> and <code>app/Modules/Discovery/</code> with actual domain code, or remove the empty directories to avoid misleading structure	HIGH RISK	MEDIUM
H5. Shared Utility Abuse	Replace <code>extract()</code> in <code>LegacyDataMapper</code> with explicit array access; register as a service via DI; consider replacing with a dedicated DTO or Transformer	MODERATE	LOW
F5. Legacy / Inconsistent Patterns	Convert <code>LegacyMonitorPoller</code> class component to function component with hooks; convert <code>LegacyDashboardWidget</code> to React Query; add Error Boundary at layout level	MODERATE	LOW

1.5 Expected Outcomes

- **Testable business logic:** Extracting `ReachabilityCalculator` , `IvrTreeBuilder` , and domain services means every business rule can be unit-tested without HTTP requests or a live database.
- **Single source of truth for KPIs:** The duplicated reachability formula converges to one canonical implementation, eliminating the risk of inconsistent alert states across API endpoints.
- **Independent domain evolution:** With repository interfaces and bounded-context boundaries enforced via `app/Modules/` , the Discovery and Connect domains can evolve, scale, and eventually extract independently.
- **Safe persistence swaps:** Repository abstractions let the team add read replicas, caching layers, or migrate from MariaDB to Aurora without touching business logic.

- **Onboarding clarity:** Removing the empty `Modules/` skeleton (or populating it) and standardizing frontend patterns (all function components, all React Query) means new developers see a consistent architecture rather than two competing paradigms.